

LLT PROJECT REPORT

Title of the Project: Railway Reservation System

Course Name: Object Oriented Design and Programming

Student Names & Register Number

RA2511026010145-DHEETCHA MS

RA2511026010162DHISHA V

RA2511026010179-RAHINI T

Class/Section-905/T1

Faculty Name- K VINOTH

Institution Name-SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

Academic Year-2025-2029

Abstract

Managing railway ticket reservations manually is time-consuming and error-prone. This project presents a **Railway Reservation System** designed using **UML diagrams** and implemented in **C++** to automate the ticket booking process. The system was first modeled using a Use Case Diagram, Class Diagram, and Sequence Diagram to clearly plan the structure and behavior before coding. The application allows passengers to **book tickets, view reservations, and cancel tickets** through a simple menu-driven interface. Each booking is assigned an auto-generated **PNR number and seat number** for easy tracking. This project demonstrates how UML-based design improves clarity, reduces errors, and bridges the gap between planning and implementation in software development.

Introduction

Background of the Problem

Railway transportation is one of the most widely used modes of travel, serving millions of passengers every day. Managing ticket reservations manually through paper-based systems or basic registers leads to problems such as double booking, lost records, long queues, and human errors. As the number of passengers grows, there is a clear need for an organized and efficient system to handle reservations smoothly.

Need for the System

A computerized Railway Reservation System eliminates the drawbacks of manual booking by automating the entire process. It ensures that each passenger gets a unique ticket with a proper PNR number and seat allocation, reduces the chance of errors, and makes it easy to view or cancel reservations at any time. Such a system saves time for both passengers and administrators while keeping records accurate and organized.

Objective of the Project

The main objectives of this project are:

- To design a Railway Reservation System using UML diagrams (Use Case, Class, and Sequence Diagrams) before implementation.
- To implement the system in C++ using object-oriented programming concepts.
- To provide basic railway reservation functionalities such as booking, viewing, and cancelling tickets.
- To demonstrate how UML-based design helps in planning and building software systematically.

Scope of the Application

This application is a console-based Railway Reservation System that covers the following:

- A passenger can book a ticket by entering their name, age, and train name.
- The system automatically assigns a unique PNR number and seat number to each booking.
- A passenger can view the list of all current reservations.
- A passenger can cancel a ticket using their PNR number.
- The system is designed for single-session use and serves as a foundational model that can be extended in the future with features like payment integration, login systems, and database connectivity.

OOP Concepts Used

Object-Oriented Programming (OOP) is a programming approach that organizes software design around data, or objects, rather than functions and logic. The Railway Reservation System is built entirely using OOP concepts in C++. Below is a detailed explanation of how each concept is applied in this project.

1. Classes and Objects

A **class** is a blueprint that defines the properties and behaviors of an entity. An **object** is a real instance created from that class.

In this project, two main classes are defined:

Reservation class — represents a single ticket booking. **RailwaySystem class** — represents the entire reservation system.

```
class Reservation {
```

```
public:
```

```
    int pnr;
```

```
    string name;
```

```
    int age;
```

```
    string train;
```

```
    int seatNo;
```

```
};
```

```
class RailwaySystem {
```

```
private:
```

```
    vector<Reservation> records;
```

```
    int nextPNR = 1000;
```

```
    int nextSeat = 1;
```

```
};
```

When the program runs, an object of `RailwaySystem` is created, which internally creates `Reservation` objects every time a ticket is booked.

```
RailwaySystem system; // Object created
```

2. Encapsulation

Encapsulation means wrapping data (attributes) and functions (methods) together inside a class and restricting direct access to internal data using access specifiers like `private` and `public`.

In this project, the `RailwaySystem` class keeps its internal data — the list of reservations, PNR counter, and seat counter — as **private**. They can only be accessed through public methods like `bookTicket()`, `viewTickets()`, and `cancelTicket()`.

```
class RailwaySystem {  
  
private:  
  
    vector<Reservation> records; // Hidden from outside  
  
    int nextPNR = 1000;  
  
    int nextSeat = 1;  
  
public:  
  
    void bookTicket(); // Controlled access  
  
    void viewTickets();  
  
    void cancelTicket();};
```

This ensures that no external code can directly modify the reservation list, protecting data integrity.

3. Abstraction

Abstraction means hiding the internal implementation details and showing only the necessary functionality to the user.

In this project, when the user selects "Book Ticket" from the menu, they simply enter their name, age, and train name. The user does not need to know how the PNR is generated, how the seat is assigned, or how the record is stored internally. All of that complexity is hidden inside the `bookTicket()` method.

```
void bookTicket() {  
  
    // User just enters details  
  
    // Internal logic handles PNR generation,  
  
    // seat assignment, and storage automatically  
  
    int pnr = nextPNR++;  
  
    int seat = nextSeat++;  
  
    records.push_back(Reservation(pnr, name, age, train, seat));  
  
}
```

The user interacts only with a simple menu, while all the complex logic runs behind the scenes.

4. Inheritance

Inheritance is the concept where one class acquires the properties and behaviors of another class. It promotes code reuse.

In this project, although inheritance is not explicitly implemented in a multi-level form, the design is structured in a way that can be easily extended. For example, the `Reservation` class can serve as a base class that can be inherited by more specific types of reservations in the future, such as:

// Future extension example

```
class TatkalReservation : public Reservation {  
  
public:  
  
    float extraCharge;  
  
    // Inherits all attributes of Reservation  
  
    // and adds its own  
  
};
```

This shows that the current design supports inheritance as a natural extension.

5. Polymorphism

Polymorphism means the ability to perform the same action in different ways depending on the context.

In this project, the `RailwaySystem` class handles three different operations — booking, viewing, and cancelling — through three separate methods. Each method processes user input differently and performs a unique action, demonstrating **functional polymorphism** through method behavior.

In a future extension, **function overloading** can be used to book a ticket in different ways:

// Future extension example

```
void bookTicket(string name, int age, string train); // Manual input
```

```
void bookTicket(int passengerID); // Auto-fetch from database
```

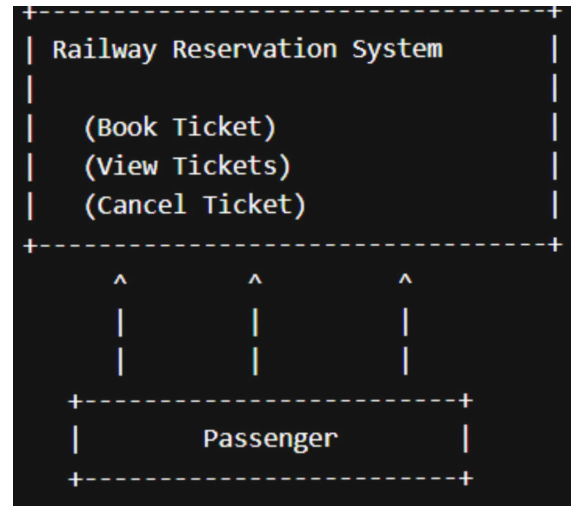
This allows the same function name to work differently based on the parameters passed.

6. System Design Using UML

UML (Unified Modeling Language) diagrams were used to plan and visualize the Railway Reservation System before implementation. Each diagram captures a different view of the system — what it does, how it is structured, and how it behaves at runtime.

a) Use Case Diagram

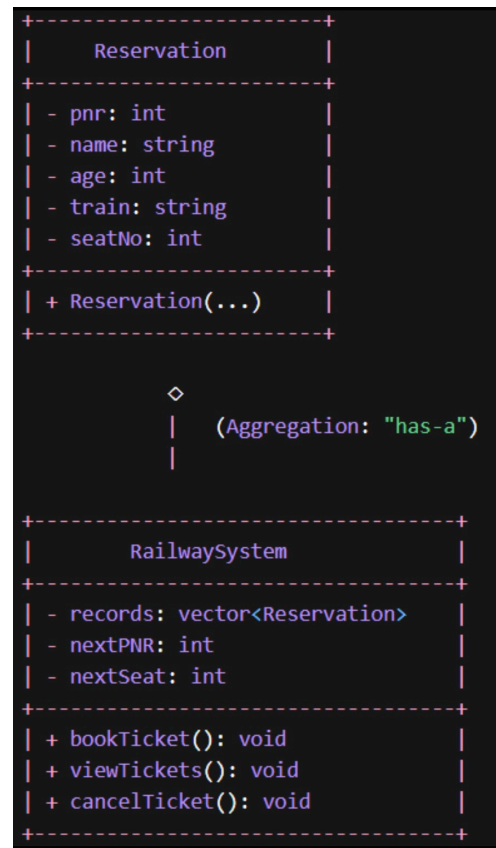
The Use Case Diagram shows the functional view of the system. It identifies the actors involved and the actions they can perform. In this system, the **Passenger** is the only actor who interacts with three main use cases: booking a ticket, viewing tickets, and cancelling a ticket



The Passenger is connected to all three use cases through simple association relationships, indicating that the passenger can directly trigger each operation.

b) Class Diagram

The Class Diagram shows the static structure of the system — the classes, their attributes, methods, and the relationships between them.

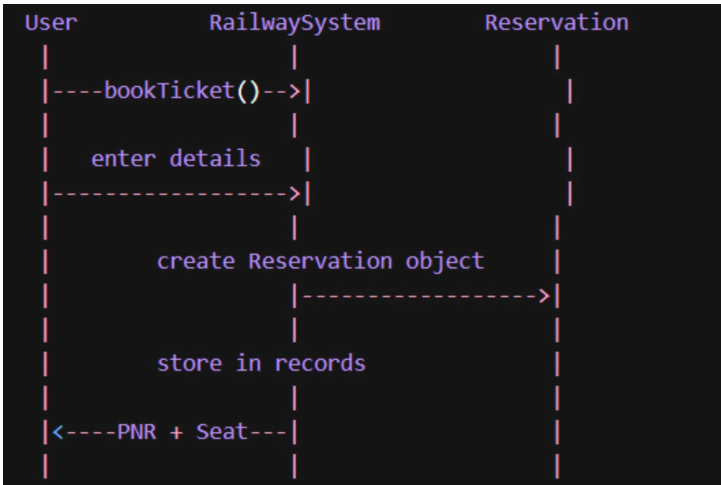


The **RailwaySystem** class has an **aggregation** relationship with the **Reservation** class, meaning it contains and manages multiple **Reservation** objects. The hollow diamond symbol (◊) represents this aggregation.

c) Sequence Diagram

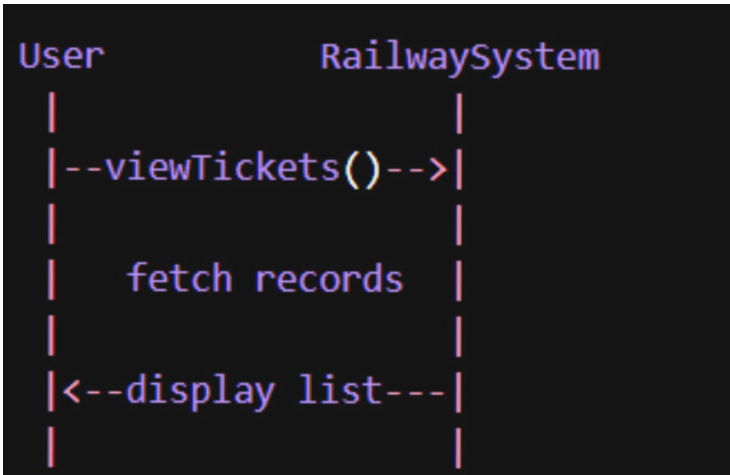
The Sequence Diagram shows the step-by-step interaction between objects over time. It captures how the User, RailwaySystem, and Reservation objects communicate during each operation.

Book Ticket:



The user initiates booking, the system collects details, creates a Reservation object, stores it, and returns the PNR and seat number.

View Tickets:



The user requests to view tickets, the system fetches all stored records and displays them.

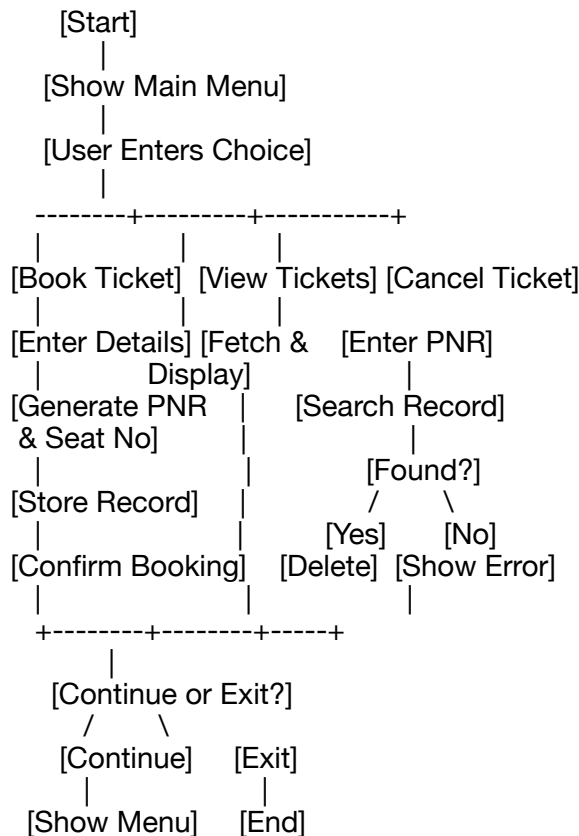
Cancel Ticket:



The user provides a PNR, the system searches the records, deletes the matching entry, and sends a confirmation.

d) Activity Diagram

The Activity Diagram shows the overall workflow of the system from start to exit, capturing the decision-making flow at each step.



The activity diagram shows how the system loops through the menu until the user chooses to exit, handling each operation through its own flow.

7. Implementation Details

Programming Language Used

The system is implemented in **C++**, a powerful object-oriented programming language that supports classes, objects, vectors, and standard input/output operations. C++ was chosen because it directly supports all OOP concepts and is well-suited for console-based applications.

Tools / IDE Used

- **IDE:** Dev-C++ / Code::Blocks / VS Code with MinGW compiler
- **Language Standard:** C++11 or higher
- **Libraries Used:** <iostream>, <vector>, <string>, <iomanip>, <cstdlib>
-

Description of Modules

The system is divided into three functional modules:

Module 1 — Reservation Data Module

This module defines the `Reservation` class, which acts as the data model for a single ticket booking. It stores all the details related to one reservation.

```
class Reservation {
public:
    int pnr;
    string name;
    int age;
    string train;
    int seatNo;

    Reservation(int p, string n, int a, string t, int s)
        : pnr(p), name(n), age(a), train(t), seatNo(s) {}
};
```

This module is responsible for holding and organizing passenger data in a structured format.

Module 2 — Data Management Module

This module defines the `RailwaySystem` class, which manages the complete list of reservations. It maintains a vector of `Reservation` objects and tracks the next available PNR and seat number.

```
class RailwaySystem {
private:
    vector<Reservation> records;
    int nextPNR = 1000;
    int nextSeat = 1;

public:
    void bookTicket();
    void viewTickets();
    void cancelTicket();
};
```

All data operations — adding, retrieving, and deleting records — are handled by this module.

Module 3 — Booking Processing Module

This module contains the logic for the three core operations of the system.

Book Ticket — collects passenger details, generates a PNR and seat number, creates a `Reservation` object, and stores it.

```

void bookTicket() {
    string name, train;
    int age;
    cin.ignore();
    cout << "\nEnter Name: ";
    getline(cin, name);
    cout << "Enter Age: ";
    cin >> age;
    cout << "Enter Train Name: ";
    cin.ignore();
    getline(cin, train);

    int pnr = nextPNR++;
    int seat = nextSeat++;
    records.push_back(Reservation(pnr, name, age, train, seat));
    cout << "Ticket Booked Successfully!\n";
    cout << "PNR: " << pnr << " | Seat No: " << seat << endl;
}

```

View Tickets — fetches and displays all stored reservations in a formatted table.

```

void viewTickets() {
    if (records.empty()) {
        cout << "\nNo reservations found.\n";
        return;
    }
    cout << "\n--- Reservation List ---\n";
    for (auto &r : records) {
        cout << r.pnr << " " << r.name << " "
            << r.age << " " << r.train
            << " " << r.seatNo << endl;
    }
}

```

Cancel Ticket — searches for a reservation by PNR and deletes it if found.

```

void cancelTicket() {
    int pnr;
    cout << "\nEnter PNR to cancel: ";
    cin >> pnr;
    for (auto it = records.begin(); it != records.end(); ++it) {
        if (it->pnr == pnr) {
            cout << "Ticket Cancelled for " << it->name << endl;
            records.erase(it);
            return;
        }
    }
    cout << "PNR not found.\n";
}

```

Module 4 — Menu Control Module

This module contains the `main()` function which drives the entire application through a menu-driven loop, taking user input and calling the appropriate module.

```
int main() {
    RailwaySystem system;
    int choice;
    do {
        cout << "\n=== Railway Reservation System ===\n";
        cout << "1. Book Ticket\n2. View Tickets\n";
        cout << "3. Cancel Ticket\n4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        switch (choice) {
            case 1: system.bookTicket(); break;
            case 2: system.viewTickets(); break;
            case 3: system.cancelTicket(); break;
            case 4: cout << "Exiting system...\n"; break;
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 4);
    return 0;
}
```

8. Output Screens / Results

This section demonstrates the working of the Railway Reservation System through sample inputs and

```
===== Railway Reservation System =====
```

```
1. Book Ticket
2. View Tickets
3. Cancel Ticket
4. Exit
```

```
Enter choice: 1
```

```
Enter Name: lily
```

```
Enter Age: 18
```

```
Enter Train Name: maharajas express
```

```
✅Ticket Booked Successfully!
```

```
PNR: 1000 | Seat No: 1
```

```
===== Railway Reservation System =====
```

```
1. Book Ticket
2. View Tickets
3. Cancel Ticket
4. Exit
```

```
Enter choice: 2
```

```
--- Reservation List ---
```

PNR	Name	Age	Train	Seat
1000	lily	18	maharajas express	1

```
===== Railway Reservation System =====
```

```
1. Book Ticket
2. View Tickets
3. Cancel Ticket
4. Exit
```

```
Enter choice: 3
```

```
Enter PNR to cancel: 1000
```

```
❌Ticket Cancelled for lily
```

```
===== Railway Reservation System =====
```

```
1. Book Ticket
2. View Tickets
3. Cancel Ticket
4. Exit
```

```
Enter choice: 4
```

```
Exiting system...
```

```
** Process exited - Return Code: 0 **
```

9. Testing

Software testing ensures that the system works correctly for all possible inputs and scenarios. The Railway Reservation System was tested manually by running the program with different inputs and verifying the outputs against expected results.

Testing Approach

Manual Testing was performed by executing the program and checking each feature individually — booking, viewing, cancelling, invalid inputs, and edge cases.

Test Case No.	Test Case Description	Input	Expected Output	Actual Output	Result
TC01	Book a valid ticket	Name: Keerthi, Age: 20, Train: Chennai Express	Ticket Booked Successfully! PNR: 1000, Seat: 1	Ticket Booked Successfully! PNR: 1000, Seat: 1	 Pass
TC02	Book a second ticket	Name: Mosmika, Age: 19, Train: Rajdhani Express	Ticket Booked Successfully! PNR: 1001, Seat: 2	Ticket Booked Successfully! PNR: 1001, Seat: 2	 Pass
TC03	Book a third ticket	Name: Thanush, Age: 21, Train: Shatabdi Express	Ticket Booked Successfully! PNR: 1002, Seat: 3	Ticket Booked Successfully! PNR: 1002, Seat: 3	 Pass
TC04	View tickets when bookings exist	Choice: 2	Display all booked tickets in table format	All tickets displayed correctly	 Pass
TC05	View tickets when no bookings exist	Choice: 2 (empty system)	 No reservations found	 No reservations found	 Pass
TC06	Cancel ticket with valid PNR	PNR: 1000	 Ticket Cancelled for Keerthi	 Ticket Cancelled for Keerthi	 Pass
TC07	Cancel ticket with invalid PNR	PNR: 9999	 PNR not found	 PNR not found	 Pass
TC08	View tickets after cancellation	Choice: 2 (after TC06)	Only remaining tickets displayed	Only Mosmika and Thanush shown	 Pass
TC09	Enter invalid menu choice	Choice: 7	Invalid choice. Please try again	Invalid choice. Please try again	 Pass
TC10	Exit the system	Choice: 4	Exiting system... Return Code: 0	Exiting system... Return Code: 0	 Pass

Total Test Cases	Passed	Failed
10	10	0

10. Features and Enhancements

10.1 Key Features Implemented

The Railway Reservation System was implemented with the following core features:

- **Ticket Booking:** Passengers can book a ticket by entering their name, age, and train name. The system automatically assigns a unique PNR number (starting from 1000) and a sequential seat number.
- **View Reservations:** All active reservations are displayed in a formatted table showing PNR, name, age, train, and seat number.
- **Ticket Cancellation:** Passengers can cancel a booking using their PNR number. The system removes the entry and confirms cancellation, or shows an error if the PNR is not found.
- **Auto-generated PNR and Seat Numbers:** Identifiers are automatically incremented with every booking, eliminating human error.
- **Input Validation:** Invalid menu choices are handled gracefully with an error message, and the program loops back to the menu without crashing.
- **Menu-driven Interface:** A simple menu is displayed at the start of every action for easy navigation.
- **OOP-based Design:** The system uses classes, encapsulation, and abstraction to separate concerns cleanly between the Reservation and RailwaySystem classes.

10.2 Possible Future Improvements

- **Database Integration:** Replacing the in-memory vector with MySQL or SQLite for persistent storage across sessions.
- **User Authentication:** A login system so passengers can access only their own reservations.
- **Multiple Trains with Seat Limits:** Support for predefined trains with fixed seat capacities per train.
- **Tatkal and Seat Class Booking:** A TatkalReservation subclass with extra charges, and seat classes like Sleeper, AC 2-Tier, and AC 3-Tier using inheritance.
- **Payment Gateway:** A payment module to simulate or process online transactions.
- **GUI:** Replacing the console interface with a Qt or web-based graphical interface.
- **File Handling for Persistence:** Using fstream to save and load reservation data between sessions.

11. Conclusion

The Railway Reservation System project provided valuable hands-on experience in applying Object-Oriented Programming principles and UML-based design to a real-world problem. Each OOP concept — classes and objects, encapsulation, abstraction, inheritance, and polymorphism — was directly applied in building a functional C++ application rather than studied in theory alone.

The use of UML diagrams before writing code proved highly effective. The Use Case, Class, Sequence, and Activity Diagrams served as a clear blueprint, reducing confusion during implementation and resulting in a cleaner, more maintainable codebase. The team observed that planning the design first leads to fewer logical errors and a better understanding of the problem.

In conclusion, this project reinforced that good software is not just about working code — it is about designing systems that are clear, extensible, and maintainable. The system, while simple in scope, reflects real-world software development practices and provides a strong foundation for future enhancements.

12. References

Books

- Balagurusamy, E. (2013). *Object Oriented Programming with C++* (6th ed.). McGraw-Hill Education.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

Websites

- GeeksforGeeks – C++ OOP Concepts: <https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/>
- TutorialsPoint – C++ Tutorial: <https://www.tutorialspoint.com/cplusplus/>
- cppreference.com – C++ Standard Library Reference: <https://en.cppreference.com/>
- Draw.io – UML Diagram Tool: <https://www.drawio.com/>
- StarUML – UML Modeling Tool: <https://staruml.io/>

Lecture Notes

- K VINOTH (2025). *Object Oriented Design and Programming – Course Lecture Notes*. SRM Institute of Science and Technology.
- SRM Institute of Science and Technology – Department of Computing Technologies, Course Material for Object Oriented Design and Programming (Academic Year 2025–2029).